



AI시대, 차세대 클라우드의 새로운 비전과 기술

openKcloud DevDay 2025

AI반도체 기반 AI SW 개발을 위한 IDE 동향

오케스트로(주)

박의규 소장





목 차

1. 서론
2. IDE 주요 동향
3. AI SW 개발 IDEs
4. 맺음말



1. 서론





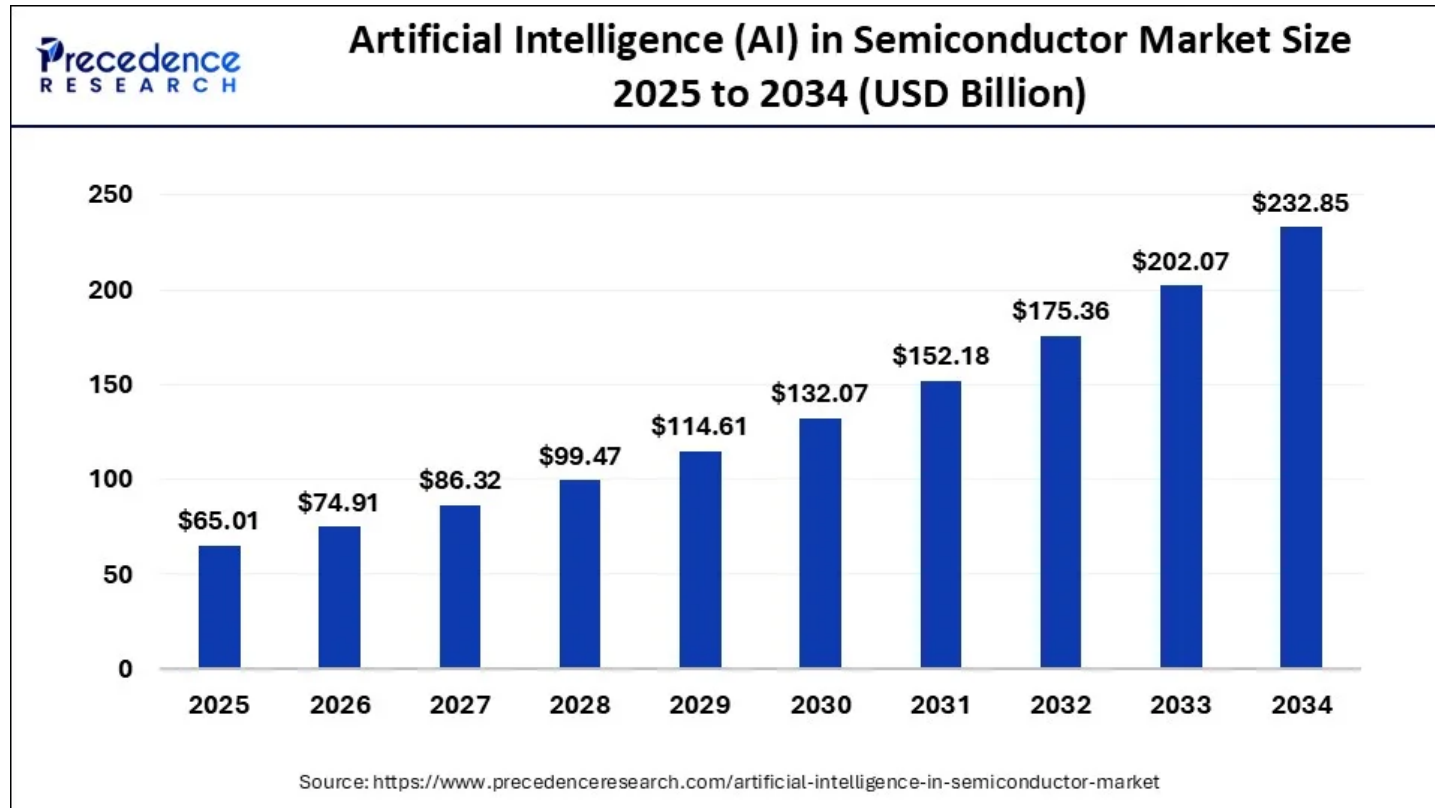
AI반도체 × AI SW 개발환경(IDE)

- 최근 McKinsey & Company는 “AI가 반도체 산업 성장의 큰 축이 되어 왔다”고 평가
- 반도체 성능만큼이나 중요한 것 → SW 개발환경
- 하드웨어 · 시스템 복잡도가 증가하면서 SW 개발도 단순한 코드 작성이 아니라 자원관리 · 추론엔진 · 배포까지 포함한 워크플로우로 진화
- AI반도체 기반 AI SW 개발에는 “단순 IDE → 반도체 자원 · 클라우드 자원 · 모델 라이프 사이클을 고려한 통합 개발환경”이 요구됨



AI반도체 시장 트렌드

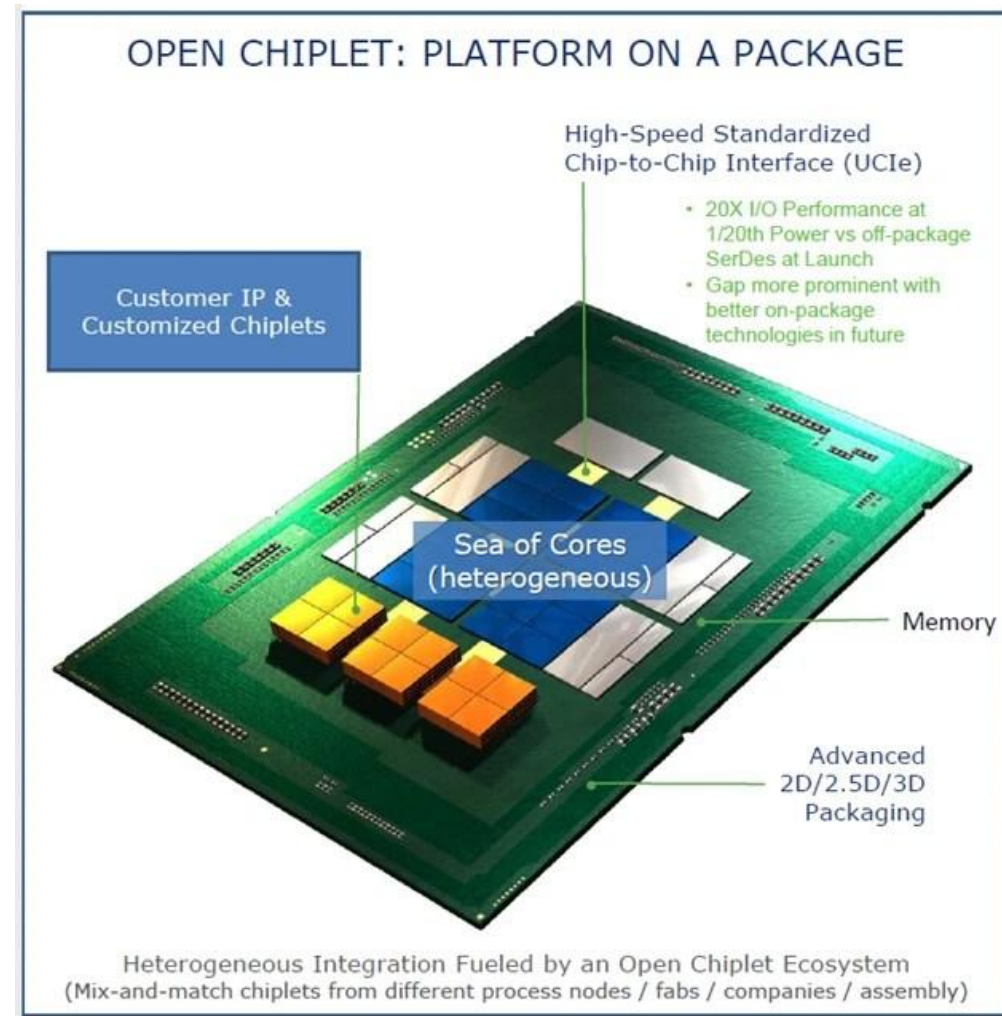
- 글로벌 “AI용 반도체 시장”은 2025년 약 650억 달러 규모에서 2034년 약 2,328억 달러 규모로 **연평균 약 15.2% 성장** 할 것으로 전망
- 반도체 산업 전반 대비 AI 관련 부문이 훨씬 빠르게 성장중이라는 평가





AI반도체 기술 트렌드

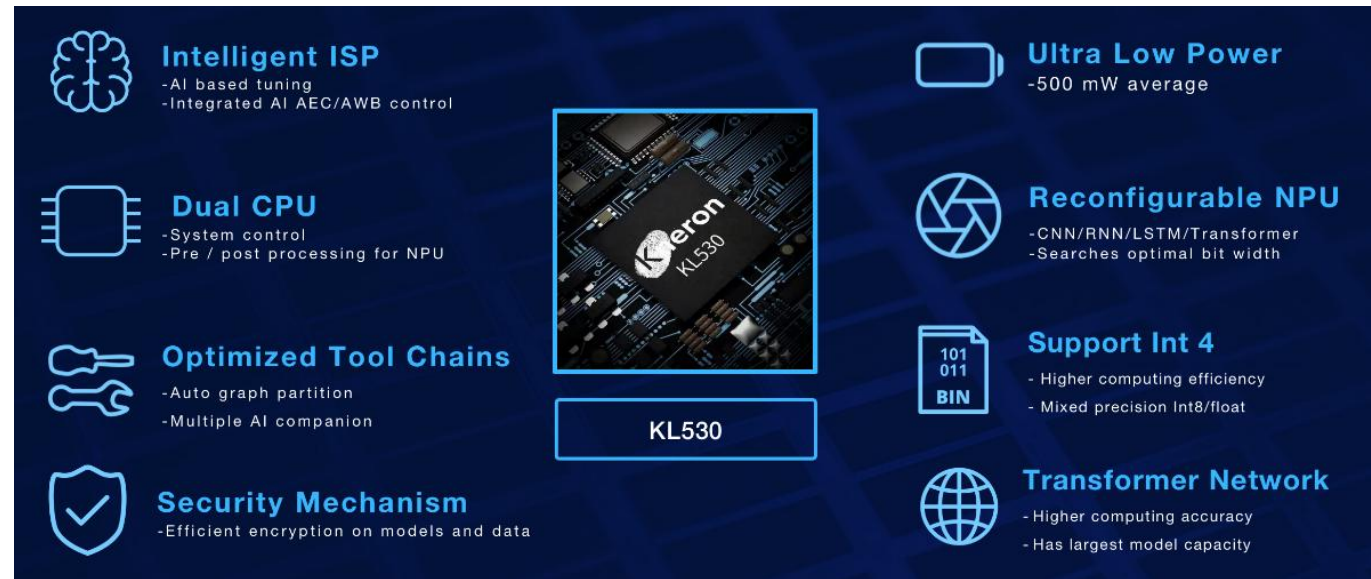
- AI 추론용 NPU / 엣지 AI용 칩 증가
- 학습(learning) 보다는 사고(thinking) · 추론(inference) 측면 칩 기술이 부상 중
- 칩렛(chiplet) 구조, 고대역폭 메모리 (HBM) 등 반도체 설계 복잡도 증가
- 소프트웨어 생태계(CUDA, AI플랫폼 등)가 하드웨어 선택에 결정적 영향





AI반도체 시장 및 기술 트렌드 시사점

- 반도체 복잡도가 증가할수록 SW 개발환경(툴, IDE, 디버깅, 프로파일링 등) 요구도 증가
- 하드웨어 제약(전력, 메모리, 실시간성, 엣지환경 등)이 SW 개발환경에 반영되어야 함
- IDE 전략 수립 시 “**하드웨어 · 소프트웨어 통합 관점**” 필수



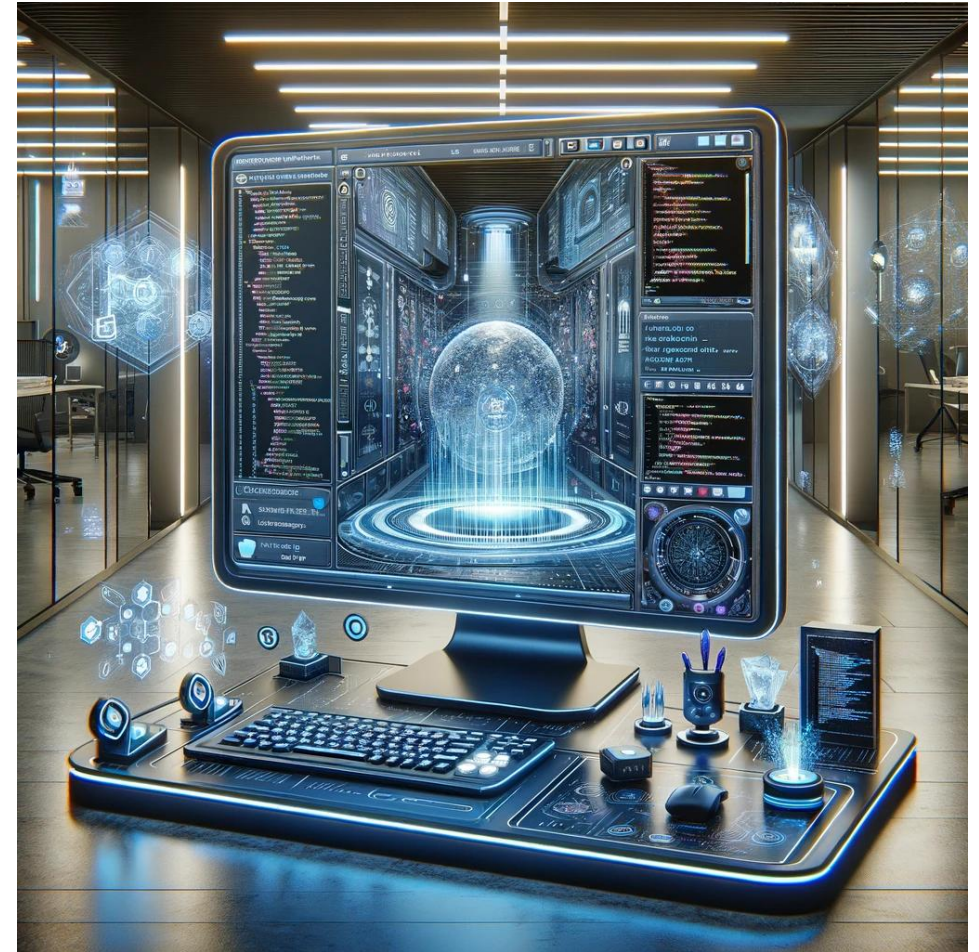
2. IDE 주요 동향





AI IDE: 단순 코드 편집기 → '에이전트형(Agentic) IDE'로 진화

- 과거 IDE는 주로 문법 강조(syntax-highlighting), 자동완성(auto-complete) 등의 기능을 제공
- 이제는 Cursor, Zed 같은 차세대 AI 편집기나 에이전트 기능을 갖춘 IDEs 등장
 - 자연어 요구사항 입력 → 코드 · 테스트 · 배포 로직 자동 생성 가능
 - 생산성 향상에도 불구하고 **자동화 편향**, **검증 오버헤드** 등의 리스크 존재
- IDE는 이제 **코드작성 도구**를 넘어 **개발자+AI 에이전트 팀워크 환경**으로 진화 중





AI IDE: 코드생성, 자동테스트, 자연어 인터페이스

• 코드 생성

- Context-aware code generation
- Natural language to code generation
- **Automatic documentation** and refactoring
- In-line explanations and debugging help
- Codebase-wide pattern recognition

• 자동 테스트

- 코드 변경 사항(diff) 기반 단위테스트, 통합테스트, E2E 테스트 자동 생성

• 자연어 인터페이스

- 개발자 입력이 자연어나 음성으로 이루어지는 **"자연어 프로그래밍(NLP→Code)"** 트렌드 등장

Top Ten AI Tools for Code Generation in 2025

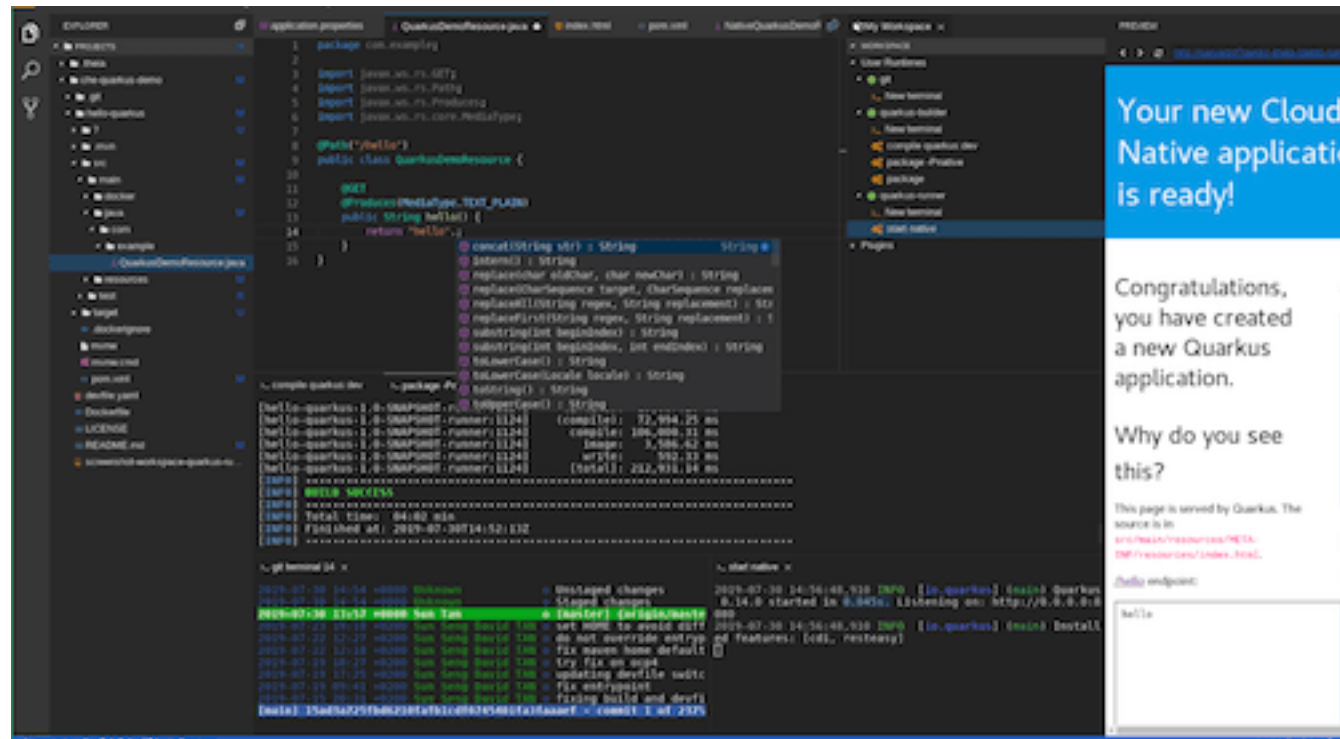


Unit Test
Generation
for Java:
[Guide](#)



클라우드/웹 기반 협업형 IDE 확산

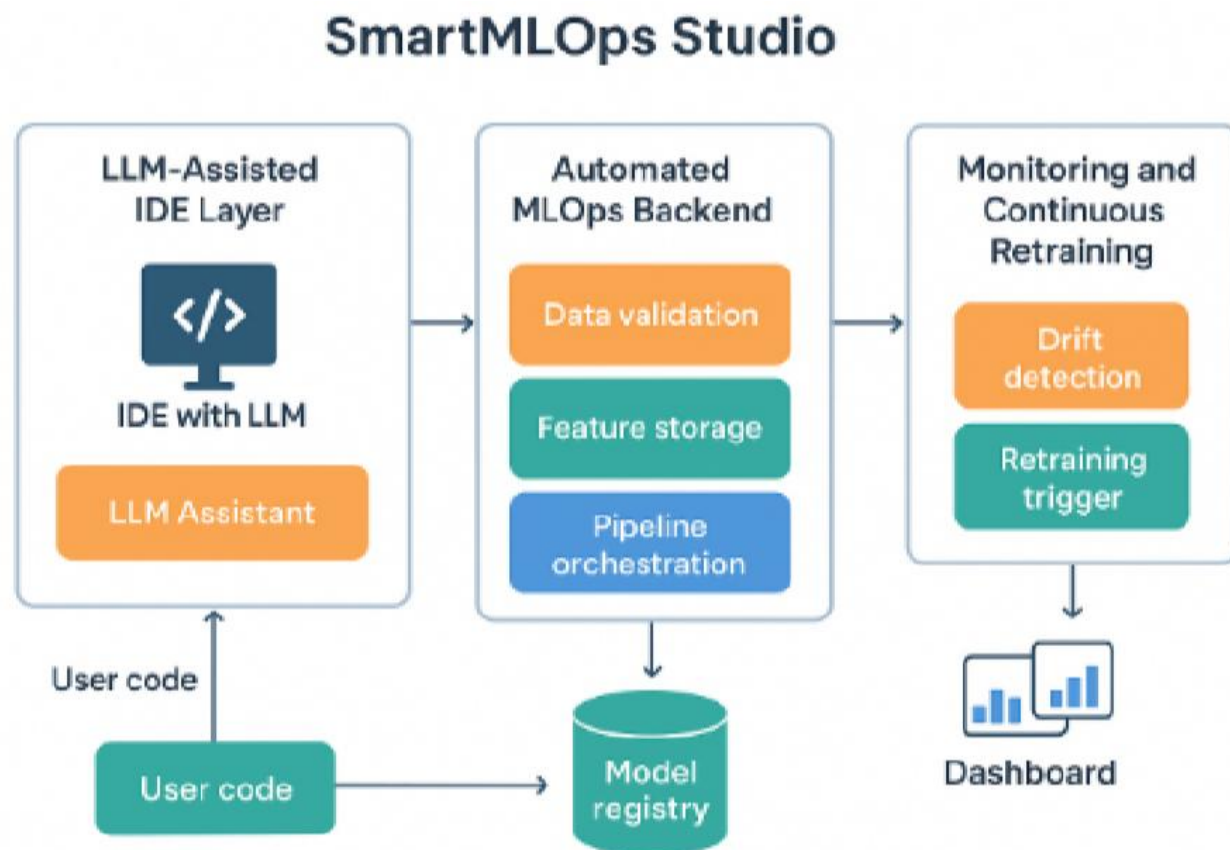
- 클라우드 네이티브 / 브라우저 기반 IDE가 개발 환경 표준으로 부상
- 특히 협업 기능 강조 : 실시간 편집, 코드 리뷰, 리소스 공유 등
- 클라우드 자원(GPU/TPU/NPU)도 IDE와 좀더 밀접하게 연계되어, 로컬이 아닌 **원격 자원을 바로 IDE 내에서 사용가능한 환경 증가**





생애주기(Lifecycle) 통합형 IDE: 모델개발→배포→모니터링

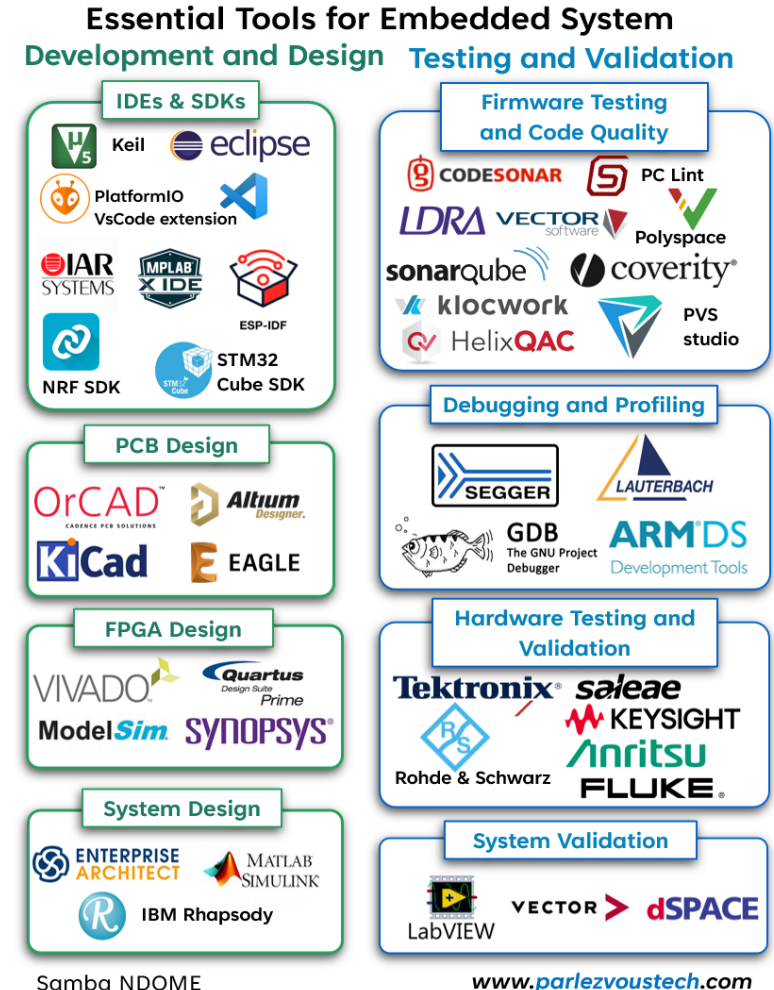
- 모델 실험, 데이터셋 관리, 배포자동화, 모니터링까지 통합
- (논문) "SmartMLOps Studio"
 - LLM-IDE 통합 제안
 - 코드 생성, 디버깅 제안, 데이터 검증, 드리프트 탐지, 재학습 트리거 등





HW·리소스 인식 IDE: 반도체·엣지·온디바이스 개발 대응

- AI 반도체가 엣지/온디바이스/데이터센터 등 다양한 타겟으로 분화됨에 따라 IDE도 단일 서버 환경 뿐만 아니라 **메모리·전력·병렬처리 제약이 있는 환경을 지원**해야 한다는 요구 증가
- **IDE 확장 플러그인 형태로** 하드웨어 가속기 SDK, 프로파일러(예: 메모리 사용량, GPU 점유율 등), 전력 모니터링 등이 포함되기 시작
- 온디바이스 개발의 경우 크로스컴파일, 임베디드 디버깅, NPU 가속 라이브러리 연계 등이 IDE 내에서 바로 가능해야 한다는 인식 증가





거버넌스·검증·보안 및 인간-AI 협업 이슈

- AI 코드 생성이 활발해지면서 코드의 **정확성, 보안성, 유지보수성에 대한 우려도 커지고 있음**
 - **자동화 편향(automation bias)**
 - **검증 오버헤드(verification overhead)**
- IDE 측면에서는 AI가 생성한 코드/테스트/설계에 대해 **사람이 리뷰·검증할 수 있는 워크플로우, 설명 가능한 AI 기능, 감사 로그 기능** 등이 요구되고 있음
- **개발자와 AI 에이전트의 협업에 어떻게 역할 분담을 하느냐**가 중요한 연구 주제로 떠오름
 - “Human-AI Experience in IDEs: A Systematic Literature Review”



IDE 동향 시사점

- IDE의 중심이 코드 편집기에서 **AI 에이전트 기반 통합환경**으로 전환 중
- **클라우드/협업/원격개발** 지원이 필수
- **AI 지원 기능**(코드 생성, 자동 테스트, 자연어 프로그래밍 등)이 빠르게 확산 중
- **생애주기(개발 → 배포 → 모니터링)** 전체 흐름을 아우르는 IDE 증가
- **하드웨어·반도체 제약**(메모리, 부하, 전력 등)을 인식하는 기능 제공

AI반도체 기반 AI SW 개발을 위한 IDE 동향

3. AI SW 개발 IDEs





AI SW 개발 IDE와 일반 SW 개발 IDE의 차이

구분	일반 SW 개발 IDE	AI SW 개발 IDE
대상	코드 로직·함수	데이터·모델 (파이프라인)
산출물	빌드된 프로그램	학습된 모델, 실험 결과
검증방법	테스트 케이스 통과 유무	성능지표 (accuracy, F1, loss)
환경관리	버전·패키지 관리	데이터·모델 버전 관리 + 환경 관리
배포	웹 서버 배포	모델 서빙(MLOps) + API 엔드포인트
협업	코드 리뷰, CI/CD	실험 공유, 모델 평가, 결과 재현 협업



VS Code (1/6)

- VS Code = “AI 개발 OS”
 - VS Code는 기본은 가벼운 에디터
 - Python, CUDA, Jupyter, Docker, Kubernetes, MLOps, LLM까지 확장 가능
 - AI SW 개발에 필요한 거의 모든 요소를 하나의 UI 안에서 처리할 수 있음
 - (Data Science) 데이터 탐색 → 모델 학습 → 시각화 → 클라우드 배포까지 수행 가능



VS Code (2/6)

- 데이터·노트북·파이프라인: Jupyter & Data Science 환경

- 네이티브 Jupyter Notebook 지원

- ✓ .ipynb 파일을 VS Code에서 바로 열고 실행 가능
- ✓ 셀 실행, Variable Explorer, Plot Viewer, Data Viewer 제공
- ✓ Python 파일에서도 # %% 셀을 이용해 "스크립트 안에 Jupyter 셀" 형태로 실행 가능

- 데이터 사이언스 전용 워크플로우 지원

- ✓ VS Code 공식 튜토리얼
- ✓ Titanic 데이터로 EDA → 전처리 → 모델 학습 → 평가까지 진행하는 튜토리얼 제공
- ✓ Python 확장 + Jupyter + Interactive Window로 처리하는 방법 설명



VS Code (3/6)

- AI 코딩 지원: Copilot & IntelliCode

- GitHub Copilot in VS Code

- ✓ VS Code에 통합된 AI 코딩 어시스턴트
 - ✓ 자연어로 "PyTorch CNN for CIFAR-10 만들어줘"처럼 지시하면, 관련 코드 전체를 생성하고, 기존 코드 컨텍스트까지 고려하여 제안함

- 다중 AI 에이전트 & Plan Mode

- ✓ GitHub는 최근 VS Code와 연계되는 Agent HQ / Plan Mode / AI 코드 리뷰 기능 발표
 - ✓ 여러 AI 코딩 에이전트를 병행 실행하고 결과 비교
 - ✓ Copilot이 "해야 할 작업 플랜"을 먼저 만들고 이를 기반으로 코드를 점진적으로 생성
 - ✓ CodeQL 등 정적 분석 도구를 이용한 AI 코드 리뷰

- IntelliCode & Pylance

- ✓ Python 편집 문서에서 AI 기반 추천을 제공하는 IntelliCode
 - ✓ 타입 추론과 고성능 언어 서버를 제공하는 Pylance



VS Code (4/6)

- 디버깅·프로파일링: Python·CUDA·원격 환경

- Python 디버깅

- ✓ 브레이크포인트, 변수 와치, 스택, 조건부 브레이크, 원격 디버깅 등
- ✓ 일반적인 디버깅 기능 + Django/FastAPI, Jupyter 등 특화 모드 지원

- Notebook 디버깅

- ✓ Notebook에서도 Run by Line, 전체 브레이크포인트 디버깅 지원
- ✓ EDA 단계에서 발생하는 미묘한 버그를 잡기 쉬움

- GPU 프로파일링 연계

- ✓ Nsight, CUDA Toolkit, Python 프로파일러와 연동하는 워크플로우 지원
- ✓ VS Code 확장 마켓플레이스: GPU 프로파일링, NVIDIA Nsight 플러그인, CUDA 언어 지원 등 제공



VS Code (5/6)

- **원격/컨테이너 개발: GPU 클러스터 & Dev Containers**
 - Remote-SSH / WSL / Dev Containers
 - ✓ Remote-SSH: 원격 리눅스 서버(예: GPU 서버)를 “로컬처럼” 열어서 개발/디버깅 가능
 - ✓ WSL: Windows에서 리눅스 기반 AI 개발 스택 사용
 - ✓ Dev Containers: devcontainer.json으로 “이 프로젝트의 표준 Docker 환경”을 정의하고, IDE가 그 컨테이너 안에서 실행되도록 함
 - GPU 드라이버·CUDA·NCCL·Python 버전이 꼬이는 문제를 컨테이너로 표준화하고, 개발자는 VS Code에서 바로 그 컨테이너/Pod 안으로 들어가 코딩·디버깅



VS Code (6/6)

- 실험 관리·MLOps 연계

- Notebook API / Data Science 확장

- ✓ Notebook API로, 다양한 Notebook 형식(예: Jupyter, custom ML notebooks)을 VS Code 안에서 열고 실행하도록 하는 확장을 만들 수 있음
 - ✓ 이를 통해 MLflow UI나 사내 전용 실험 도구를 IDE에 녹여 넣는 것이 가능

- (예시) MLflow/W&B Stack

- ✓ 확장 또는 간단한 Custom Webview로
 - 실험 리스트, 메트릭 그래프, 아티팩트 목록을 보여주고, “이 실험 다시 실행” 같은 버튼을 IDE에 배치 가능



Eclipse Theia (1/6)

- “AI용 전용 IDE를 직접 만들 수 있는 틀”
- Theia는 완성된 IDE 제품이 아니라, 브라우저/데스크톱에서 동작하는 IDE를 만들 수 있는 **오픈소스 프레임워크**
- Eclipse 재단 산하 프로젝트이고, 완전히 **벤더 중립(vendor-neutral)** 구조 + 거버넌스
- “Theia IDE”라는 패키징된 제품도 있지만, 이 역시 **Theia 프레임워크에 플러그인·확장을 묶어서 배포한 래핑**



Eclipse Theia (2/6)

- Cloud & Desktop 하이브리드
 - Theia 자체가 웹 기술 기반(HTML/TypeScript)
 - 브라우저에서 클라우드 IDE로 실행 가능
 - Electron 기반 데스크톱 IDE로도 패키징 가능
 - Gitpod 같은 Kubernetes 기반 클라우드 개발환경(CDE)의 핵심으로 사용됨
- Monaco + LSP + DAP: 언어·디버거 확장 구조
 - Theia는 VS Code와 마찬가지로 Monaco Editor, LSP(Language Server Protocol), DAP(Debug Adapter Protocol) 활용 가능
 - LSP로 언어 기능(자동완성, 오류 체크, 리팩토링), DAP로 디버깅 기능을 연동하는 구조
- Python, C/C++, CUDA, Rust, Go, JS 등 언어 서버를 연동하기 쉬워서
 - PyTorch/TensorFlow, CUDA 커널, AI 서비스용 백엔드까지 하나의 IDE 안에서 활용 가능



Eclipse Theia (3/6)

• Theia Extension + Plugin + VS Code Extension

➤ Theia Extensions

- ✓ Theia 앱을 빌드할 때 포함하는 **컴파일 타임 확장**
- ✓ IDE의 코어 동작/레이아웃/서비스를 깊이 있게 바꾸는 용도

➤ Theia Plugins

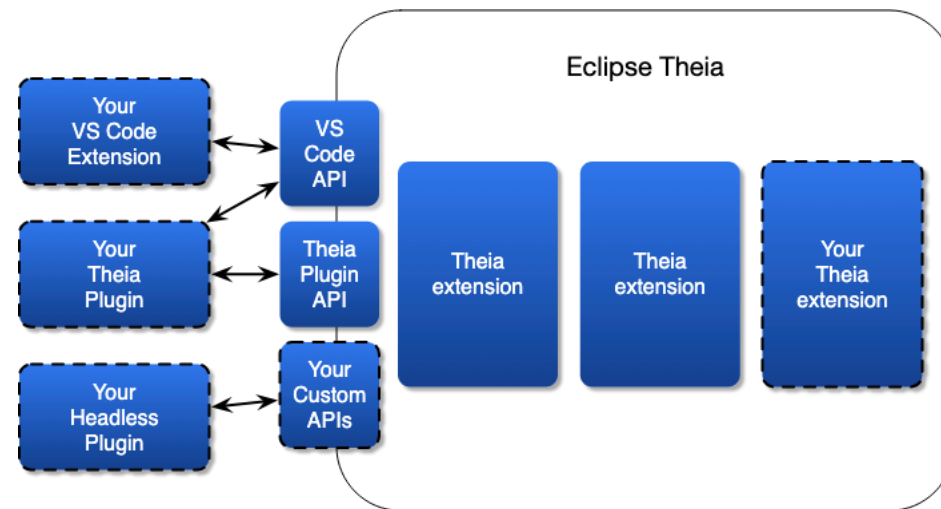
- ✓ 런타임에 구동되는 플러그인 (headless 모드도 가능)
- ✓ VS Code Extension이 돌아가는 것과 비슷한 모델

➤ Headless Plugins

- ✓ UI 없이 백그라운드에서만 돌면서 작업하는 플러그인
- ✓ 실험 기록, 자원 사용 로깅, 정책 검사 등

➤ VS Code Extensions 호환

- ✓ OpenVSX 레지스트리를 통해 VS Code 확장을 직접 설치 가능
- ✓ Python, Jupyter, Docker, Kubernetes, GitLens, 일부 ML/AI 관련 확장 등 대부분 사용 가능





Eclipse Theia (4/6)

- 브라우저 기반 AI 개발환경: GPU 클러스터와의 자연스러운 연계
 - Gitpod/Codeanywhere 같은 서비스들이 Theia 기반 클라우드 IDE를 제공
 - Kubernetes + 컨테이너 + VS Code/Theia 조합을 사실상 표준으로 만들고 있음
 - AI SW 개발 적용
 - ✓ 사용자가 Git repo 선택
 - ✓ K8s에서 GPU 자원이 붙은 Pod/Workspace 생성
 - ✓ Pod 안에서 Theia가 구동하여 Python/Conda/Poetry, Jupyter, CUDA/드라이버 등이 이미 준비된 상태로 바로 실험/개발 시작 가능
 - “개발환경 세팅”을 거의 없애고, 매번 동일한 GPU/라이브러리 스택으로 개발 가능
 - 쿠버네티스·GPU 스케줄링·멀티 테넌시·보안 정책을 인프라·플랫폼 레벨에서 제어 가능



Eclipse Theia (5/6)

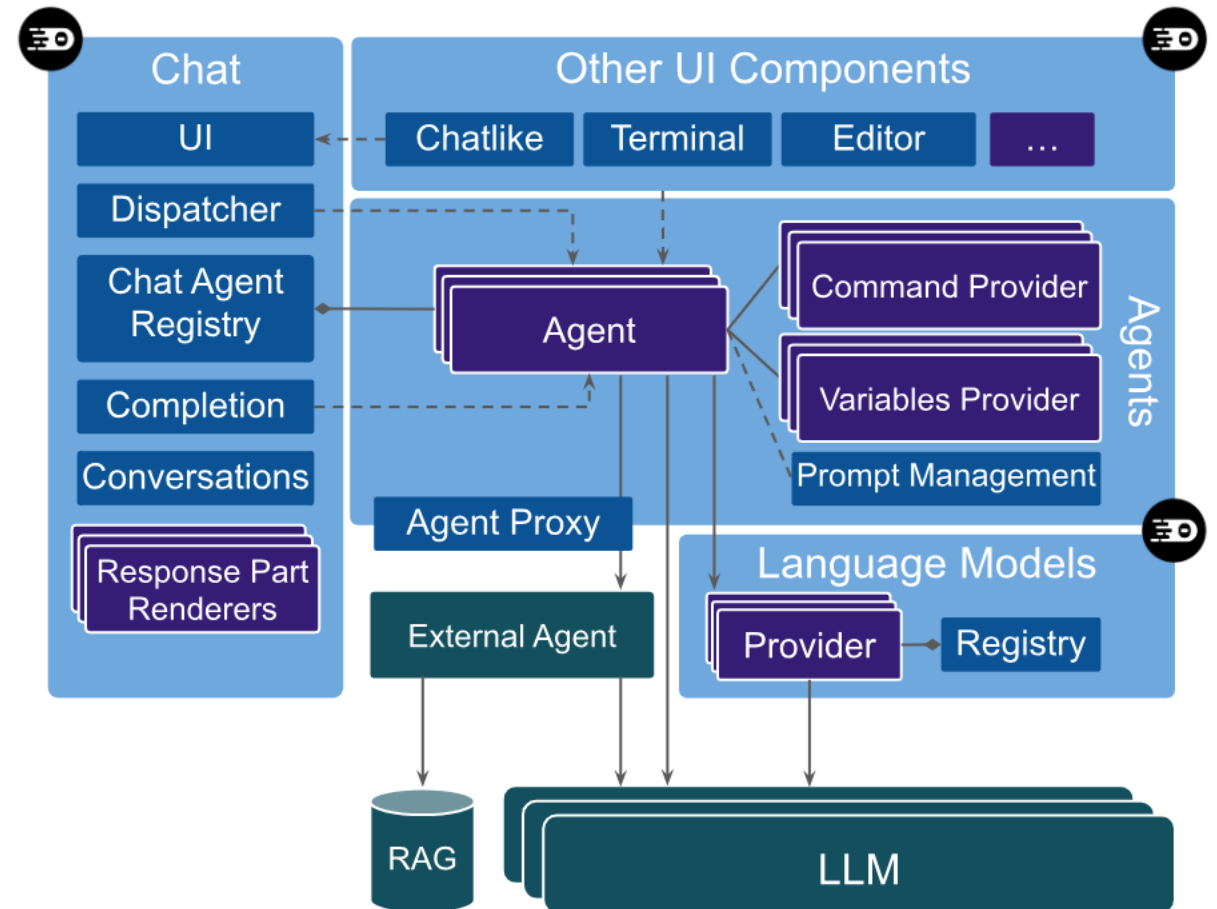
- Jupyter & Data Science 워크플로우

- Theia는 자체적으로도 Notebook 지원이 가능하고, VS Code의 Jupyter/Python 확장 상당수를 사용이 가능함
- Theia IDE 내에서 데이터 EDA, 전처리, 모델 학습/평가, 실험 기록 코드 등을 **노트북+스크립트 조합**으로 운영 가능
- MLflow / W&B 웹 UI를 iframe 처럼 임베드하거나, 확장으로 전용 패널을 적용하면 “**AI 프로젝트 전주기**”를 한 화면에서 관리 가능



Eclipse Theia (6/6)

- AI/LLM 통합: **Theia AI 프레임워크**
- Eclipse 재단에서는 2024년부터 **"Theia AI"** 라는 이름으로 AI 통합을 위한 서브프레임워크 준비
- **AI-driven IDEs and tools**
 - 코드 중심 IDE뿐 아니라, 모델·데이터·도메인 툴에 AI를 심을 수 있는 공통 프레임워크
 - 코드 어시스턴트, 자연어로 IDE 명령, 분석/시각화 도구에 AI 설명/요약 기능 추가 등





JetBrains (1/4)

- “AI 개발용 올인원 정통 IDE 계열”
- PyCharm: AI 애플리케이션/서비스 개발 중심
- DataSpell: 데이터/모델 실험 및 분석 중심
- AI Assistant: LLM 코파일럿



JetBrains (2/4)

- **PyCharm: AI 애플리케이션 개발의 “정석 IDE”**

- **Python + 데이터 과학 + Jupyter 통합**

- ✓ PyCharm 프로 버전은 과학 컴퓨팅/데이터 시각화/ML 기능 내장
- ✓ Jupyter Notebook을 IDE 안에서 열고 편집·실행·디버깅 가능

- **데이터 과학 기능**

- ✓ NumPy, pandas, Matplotlib, Plotly 등 과학 라이브러리 지원
- ✓ 전용 “과학 툴 윈도우”: 변수/배열 뷰어, 그래프/표 시각화, 실험 중간결과 확인 용이
- ✓ 작은 단일 모델/데이터 분석은 별도 툴 없이 PyCharm 하나로 충분히 처리 가능

- **환경·컨테이너·원격 개발 (GPU 환경 포함)**

- ✓ PyCharm 원격 개발(Remote Development) 기능 제공
- ✓ 리모트 머신, 컨테이너(Dev Container, Docker), WSL 등의 환경에 IDE 백엔드를 띄우고, 로컬 클라이언트에서 접속해 개발하도록 지원
- ✓ Docker를 인터프리터로 쓰도록 설정할 수 있어, 특정 Docker 이미지(예: CUDA + PyTorch + 드라이버)에서만 실행가능한 구조 생성 가능



JetBrains (3/4)

- DataSpell: “데이터·모델 실험”에 특화된 JetBrains IDE
 - DataSpell은 **탐색적 데이터 분석(EDA)**과 **ML 모델 프로토타이핑**에 특화된 IDE
 - JetBrains는 “**데이터 과학자와 소프트웨어 개발자는 원하는 IDE UX가 다르다**”는 피드백을 바탕으로, PyCharm 엔진을 활용해 별도의 데이터 과학 전용 IDE로 분리
 - ✓ PyCharm = **개발자용**, DataSpell = **데이터 과학자/ML 연구자용**
 - 기본 뷰가 노트북 중심이며, 파일 시스템 + 노트북 + 코드 모듈을 하나의 워크스페이스에서 관리
 - **Python + SQL + 데이터베이스 연동**
 - ✓ 데이터 쿼리 → 바로 프레임/플롯으로 확인
 - **Data Wrangler** 기능
 - ✓ 데이터 정리·변환 작업을 인터랙티브하게 도와주는 UI (파이썬 코드도 동시에 생성)



JetBrains (4/4)

- **JetBrains AI Assistant: IDE 전반의 “AI 코파일럿”**
 - 컨텍스트 인식 코드 생성·자동완성
 - 기존 코드 설명, 복잡한 함수/파일 구조 요약
 - 리팩토링 제안, 버그 후보 지점 설명
 - 테스트 코드 자동 생성
 - 커밋/PR 변경사항 요약
 - Jupyter 셀/데이터프레임에 대한 AI 설명·변환·시각화 추천



AI SW 개발 IDEs 시사점

- VS Code : 빠른 시작, **폭넓은 에코시스템** 제공
- Theia : 조직 맞춤형, **GPU 클러스터 + 클라우드 기반 개발환경** 제공
- JetBrains : 고도화된 **Python/ML + 데이터 분석 + 서비스 통합 환경** 제공
- **고려사항**
 - GPU/NPU 자원 인식 기능 (메모리·전력·대역폭 등)
 - AI 어시스턴트/코딩 자동화 기능
 - 라이선스/비용
 - 팀 규모/클러스터 운영 구조(예: K8s GPU Operator)

AI반도체 기반 AI SW 개발을 위한 IDE 동향

4. 맺음말





IDE Wars



출처: *The IDE Wars of 2025 Choosing Your Weapon in the Battle for Developer Dominance*



Metrics for Evaluating an IDE

- **Speed & Agility**: The Need for Swifttness
- **Adaptability**: Thriving in a Dynamic Landscape
- **Language Mastery**: Speaking the Language of Code
- **AI Augmentation**: Your AI Ally
- **Defensive Capabilities**: Shielding Against Bugs
- **Teamwork & Alliances**: Coding in Concert
- **Community & Support**: Your Lifeline in the Arena



The Future of IDEs

- **The Rise of the Machines**: AI Ascendant
- **The Cloud Kingdom**: The Reign of the Cloud-Native IDE
- **The Convergence**: The All-in-One Development Powerhouse
- **The Fortress of Security**: Shielding the Code Kingdom



맺음말

Do you Code like a pro? Great.

Or do you Code like a noob? Fine, that's great too.

But if you don't know your IDE, you're just making things harder on yourself.
Learning to code is challenging enough.

Understanding your IDE is like having a map of the coding world – it'll save you from getting hopelessly lost.

Choose your IDE, hone your skills, and adapt to the ever-changing arena.

The fate of your code depends on it.

Good luck.

감사합니다.

